



Curso Javascript básico

Aula 05 - Trabalhando com Arrays



Trabalhando com Arrays

propriedade - length

.length

A propriedade “length” retorna um valor inteiro (number) com a quantidade de elementos de um array.

```
var array1 = ['branco', 10, 20, 30, null];  
array1.length;  
console.log(array1); // resultado será 5
```



Trabalhando com Arrays

Métodos - concat()

.concat()

O método “concat()”, serve para concatenar(juntar) dois arrays.

```
var array1 = ['branco', 10, 20, 30, null];  
var array2 = ['verde', 40, 50, 60, undefined];  
array1 = array1.concat(array2);  
console.log(array1);
```

O resultado desta operação será um array com os elemento dos dois arrays concatenados.

```
["branco", 10, 20, 30, null, "verde", 40, 50, 60, undefined]
```



Trabalhando com Arrays

Métodos - join()

.join()

O método “join()” agrupa os elementos de um array e mostra no formato de string.

```
var array1 = ['branco', 10, 20, 30];  
array1 = array1.join('-');  
console.log(array1);
```

O resultado da operação puxou as propriedades do *array1* e listou separando com um traço.

branco-10-20-30



Trabalhando com Arrays

Métodos - push()

.push()

O método “push()” serve para adicionarmos elementos no final do array.

```
var array1 = ['branco', 10, 20, 30];  
array1.push('novo elemento');  
console.log(array1);
```

O resultado nos trouxe um novo elemento no final do *array1*

```
['branco', 10, 20, 30, "novo elemento"]
```



Trabalhando com Arrays

Métodos - pop()

.pop()

O método “pop()” remove o último elemento de um array.

```
var array1 = ['branco', 10, 20, 30, "novo elemento"];  
array1.pop();  
console.log(array1);
```

Veja que o último elemento do *array1* foi removido.

```
['branco', 10, 20, 30]
```



Trabalhando com Arrays

Métodos - shift()

.shift()

O método “shift()” é o contrário do “pop()”, ele remove o PRIMEIRO elemento do array.

```
var array1 = ['branco', 10, 20, 30, "novo elemento"];  
array1.shift();  
console.log(array1);
```

Primeiro elemento do array1 removido com sucesso.

```
[10, 20, 30, "novo elemento"]
```



Trabalhando com Arrays

Métodos - unshift()

.unshift()

O método “unshift()” faz o contrário do que o “push()” faz, ele adiciona um elemento no início de um array.

```
var array1 = ['branco', 10, 20, 30, "novo elemento"];  
array1.unshift('novo elemento');  
console.log(array1);
```

Veja o resultado impresso no console.

```
["novo elemento", "branco", 10, 20, 30, "novo elemento"]
```



Trabalhando com Arrays

Métodos - slice()

.slice()

O “slice()” extrai uma sessão de um array e retorna um novo array, veja:

```
var array1 = ['branco', 10, 20, 30, "novo elemento"];  
var novoArray = array1.slice(1,3);  
console.log(novoArray);
```

Acima estamos pedindo para que o *array1* seja selecionado do elemento 1 até o elemento 3 para gerarmos um novo array somente com os elementos que queremos.

[10, 20]



Trabalhando com Arrays

Métodos - splice()

.splice()

No método “splice()” podemos escolher um índice inicial e final para substituírmos valores entre eles.

```
var array1 = ['branco', 10, 20, 30, "novo elemento"];  
array1.splice(1,2, "splice");  
console.log(array1);
```

No programa acima o comando para o **splice()** adicionar um elemento novo no lugar dos elementos 1 e 2, veja o resultado:

```
["branco", "splice", 30, "novo elemento"]
```



Trabalhando com Arrays

Métodos - reverse()

.reverse()

O método “reverse()” inverte a ordem dos elementos do array.

```
var array1 = ['branco', 10, 20, 30, "novo elemento"];  
array1.reverse();  
console.log(array1);
```

E o resultado no console é.

```
["novo elemento", 30, 20, 10, "branco"]
```



Trabalhando com Arrays

Métodos - toString()

. toString()

O método “toString()” converte um array em uma string de valores(separados por vírgula)..

```
var array1 = [" branco", 10, 20, 30, "novo elemento"];  
array1.toString();  
console.log(array1);
```

E o resultado no console é.

branco, 10, 20, 30, novo elemento



Trabalhando com Arrays

Métodos - from()

.from()

O método “from()” é utilizado quando queremos criar um array a partir de um objeto que não é um array. Um exemplo são as strings em que cada caractere é convertido para um elemento de array.

```
var frase = "JavaScript"  
var arrayFrase = Array.from(frase);  
console.log(arrayFrase); // retorna [ 'J', 'a', 'v', 'a', 'S', 'c', 'r', 'i', 'p', 't' ]
```



Trabalhando com Arrays

Métodos - isArray()

.isArray ()

O método isArray () é empregado quando queremos descobrir se uma variável ou constante é um objeto array. Caso o resultado seja positivo, ele retorna o valor verdadeiro. Se for negativo, o retorno é falso. Veja o exemplo abaixo.

```
var nomes = ["João", "Maria", "Paulo", "Pedro"];  
console.log(Array.isArray(nomes)); // resulta em true
```

```
var nomesString = "João, Maria, Paulo, Pedro";  
console.log(Array.isArray(nomesString)); // resulta em false
```



Trabalhando com Arrays

Métodos – indexOf(), lastIndexOf() e includes().

Os métodos `indexOf`, `lastIndexOf` e `includes` possuem a mesma sintaxe e fazem essencialmente o mesmo que suas contrapartes de string, mas operam em itens em vez de caracteres. Vejamos a definição de cada um:

.indexOf() - Procura no array por um elemento e retorna o índice onde foi encontrado. Se não encontrar retorna -1;

.lastIndexOf() - Exatamente como o anterior. A diferença é que ele inicia a partir do fim do array;

.includes() - Procura no array pelo elemento a partir do índice e retorna `true` se encontrar e `false` se não encontrar. É usado para verificar se um objeto existe em um array;



Trabalhando com Arrays

Métodos – `indexOf()`, `lastIndexOf()` e `includes()`.

```
var bau = [1, 0, false, "banana"];
```

.indexOf()

```
console.log ( bau.indexOf("banana") ); // retorna 3 (que é seu índice no array)
```

.lastIndexOf()

```
console.log ( bau.lastIndexOf("banana") ); // retorna 3 (que é seu índice no array)
```

.includes()

```
console.log ( bau.includes("banana") ); // retorna true pois achou o elemento
```



Trabalhando com Arrays

Métodos - forEach()

.forEach()

Caso você precise executar algum código para cada elemento do Array, executar um `forEach` é muito mais simples do que criar um `for` ou `while`, já que não precisamos declarar variáveis de controle.

```
const numbersList = [1, 2, 3, 4, 5];  
numbersList.forEach((number, index, array) => {  
  myList += number;  
});
```

O `.forEach()` irá jogar cada um dos elementos do Array no primeiro parâmetro, o índice do elemento no segundo e o Array original no terceiro.



Trabalhando com Arrays

Métodos - map()

.map()

Nós utilizamos o .map() quando queremos fazer alguma modificação nos elementos de um Array.

```
const usersList = [  
  {name: 'João', credit: 500},  
  {name: 'Maria', credit: 800}  
];  
const newUsersList = usersList.map((user, index, array) => {  
  user.credit += 100;  
  return user;  
})
```

No exemplo acima alteramos o valor de uma propriedade de cada um dos elementos do Array. Na última linha nós precisamos executar o comando return para indicar o que será retornado para o Array. No final teremos um novo Array.



Trabalhando com Arrays

Métodos - filter()

.filter()

Como o próprio nome indica, serve para filtrarmos os elementos de um Array. Passamos para ele uma função. Se essa função retornar true, o elemento será inserido no novo Array que será criado. Se a função retornar false, o elemento será ignorado.

```
const usersList = [
  {name: 'João', credit: 600},
  {name: 'Maria', credit: 900},
  {name: 'Carlos', credit: 300},
  {name: 'Vanessa', credit: 200},
];

const newUsersList = usersList.filter((user, index, array) => user.credit > 500);
/* Resultado:
[
  {name: 'João', credit: 600},
  {name: 'Maria', credit: 900}
]
*/
```



Trabalhando com Arrays

Métodos - find()

.find()

Usamos esse método quando queremos encontrar algum elemento dentro no Array. Para isso, passamos uma função que irá retornar true ou false. O primeiro true que for retornado irá finalizar a função e retornar o elemento em que estamos.

```
const usersList = [  
  {name: 'João', credit: 600},  
  {name: 'Maria', credit: 900},  
  {name: 'Carlos', credit: 300},  
  {name: 'Vanessa', credit: 200},  
];  
  
const carlos = usersList.find((user, index, array) => user.name === 'Carlos');  
/* Resultado:  
   {name: 'Carlos', credit: 300}  
*/
```



Trabalhando com Arrays

Métodos - `findIndex()`

`.findIndex()`

Faz o mesmo que o `.find()`, mas retorna o índice do elemento encontrado ao invés de retornar o próprio elemento.

```
const userList = [  
  {name: 'João', credit: 600},  
  {name: 'Maria', credit: 900},  
  {name: 'Carlos', credit: 300},  
  {name: 'Vanessa', credit: 200},  
];  
  
const carlos = userList.findIndex((user, index, array) => user.name === 'Carlos');  
/* Resultado:  
   2  
*/
```



Trabalhando com Arrays

Métodos - every()

`.every()`

Serve para testarmos se todos os elementos do Array passam em uma condição. Passamos uma função que retorna true ou false. Se todos os retornos forem true, significa que todos os elementos passaram no teste, e a função retornará true.

```
const userList = [  
  {name: 'João', credit: 600},  
  {name: 'Maria', credit: 900},  
  {name: 'Carlos', credit: 300},  
  {name: 'Vanessa', credit: 200},  
];
```

```
const result1 = userList.every((user, index, array) => user.credit < 1000);  
const result2 = userList.every(user => user.credit < 500);
```



Trabalhando com Arrays

Métodos - some()

.some()

O .some() faz algo parecido com o .every(). A diferença é que o .every() só retorna true se todos os elementos passarem no teste. O .some() retorna true se pelo menos um elemento do Array passar no teste.

```
const usersList = [  
  {name: 'João', credit: 600},  
  {name: 'Maria', credit: 900},  
  {name: 'Carlos', credit: 300},  
  {name: 'Vanessa', credit: 200},  
];  
  
const result = usersList.some((user, index, array) => user.credit === 300);
```

Verificamos se há pelo menos um usuário com crédito igual a 300. O resultado será true.



Trabalhando com Arrays

Métodos - sort()

`.sort()`

O `.sort()` serve para ordenar os elementos de Arrays. Muitas pessoas utilizam este método de maneira errada, apenas executando `array.sort()`. Isso pode causar retornos inesperados, pois os elementos serão convertidos em texto.

O correto é passar uma função que compare dois elementos. Assim, podemos ordenar um Array com qualquer tipo de objeto, ordenando por qualquer propriedade.



Trabalhando com Arrays

Métodos - sort()

```
const numbersList = [4, 5, 7, 8, 2];  
const orderedList = numbersList.sort((a, b) => {  
  if(a < b){  
    return -1;  
  } else if(a > b){  
    return 1;  
  }  
  return 0;  
})
```



Trabalhando com Arrays

Métodos - reduce()

.reduce()

Esta função serve para reduzirmos o conteúdo de um Array para apenas um elemento. O exemplo mais clássico é somar todos os valores de um Array.

```
const numbersList = [1, 2, 3];  
const total = numbersList.reduce((total, currentElement) => total + currentElement)  
/* Resultado:  
    6  
*/
```